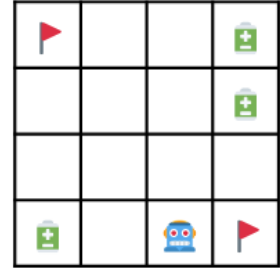


Q1)

Consider a delivery drone moving on a $N \times N$ grid.

- At each time step, the drone must move to an adjacent square (up, down, left, right). All actions cost 1.
- There are k fixed destination squares on the grid. The drone needs to visit all k destinations at least once.
- The drone starts with a battery charge of C units, and must always have at least 1 unit of charge at every time step. The drone loses 1 unit of charge per action.
- There are p power cells on the grid (with fixed locations). When the drone visits a power cell, its battery charge resets to C units, and the power cell cannot be used again.



Sample grid with $N = 4$,
 $k = 2$ destinations ,
 $p = 3$ power cells ,
 and drone .

What is the maximum branching factor for this problem?

4

What is the size of the smallest state space representation for this problem?

The drone's position can be one of N^2 squares.

At any point in time, any combination of the p power cells and k destination squares will be visited, so we need one Boolean per power cell and one Boolean per destination square.

The drone's current battery charge will be one of C different values (it can never be empty).

This gives a size of: CN^22^{p+k}

Algorithms that are guaranteed to find the lowest-cost solution for a search problem like this?

- | | |
|---|--|
| ☐ A DFS tree search | ☐ E UCS tree search |
| ☐ B DFS graph search | ☐ F UCS graph search |
| ☐ C BFS tree search | ☐ G None of the above |
| ☐ D BFS graph search | |

DFS is not guaranteed to find the lowest-cost solution.

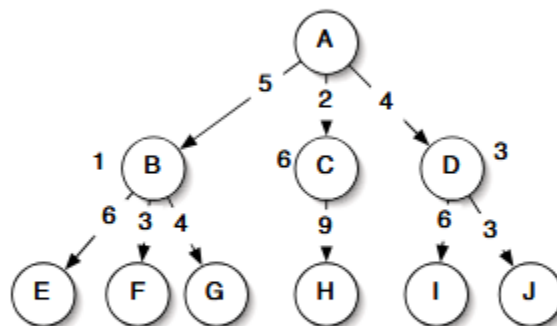
BFS will find the lowest-cost solution, since all actions cost 1.

UCS will find the lowest-cost solution in general.

Both tree and graph search work, because the solution is finite-length (the length of the solution is limited by the battery charge, so it can't be infinite).

Q2)

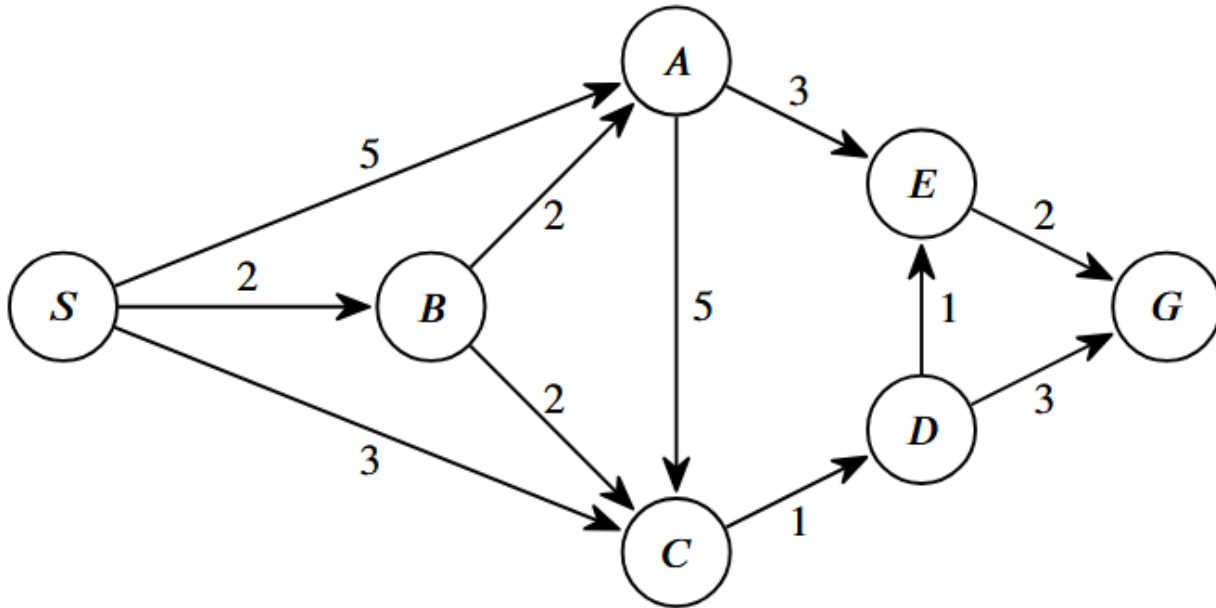
Consider the tree shown below. The numbers on the arcs are the arc lengths; the numbers near states B, C, and D are the heuristic estimates; all other states have a heuristic estimate of 0.



Assume that the children of a node are expanded in alphabetical order when no other order is specified by the search, and that the goal is state *J*. No visited or expanded lists are used. What order would the states be expanded by each type of search. Write only the sequence of states expanded by each search.

Search Type	List of states
Breadth First	A B C D E F G H I J
Depth First	A B E F G C H D I J
Progressive Deepening Search	A A B C D A B E F G C H D I J
Best-First Search	A B E F G D I J
A* Search	A B D J

Q3)



What will be the path returned by **DFS Graph Search**?

$S \rightarrow C \rightarrow D \rightarrow G$

We start by putting just the start state on the fringe:

Fringe: { S }

We pop the most-recently-added item off the stack, namely S.

Fringe: { S }

Then, we add all of S's successors onto the stack:

Fringe: { SA SB SC }

Per the hint, we added SA, SB, SC, in that order. Since we added SC most-recently, it is the next item popped off the stack.

Fringe: { SA SB SC }

Then, we add all of SC's successors onto the stack:

Fringe: { SA SB SCD }

The most-recently-added item on the stack is SCD, so we pop SCD off the stack next.

Fringe: { SA B SCD }

Then, we add all of SCD's successors onto the stack:

Fringe: { SA SB SCDE SCDG }

Per the hint, we added SCDE and SCDG in that order. Since SCDG was added most recently, it is the next item popped off the stack.

SCDG reaches the goal, and we've popped this solution off the stack, so we are done.

What will be the path returned by **BFS Tree Search**?

We start by putting just the start state on the fringe:

Fringe: { S }

We pop the oldest item off the queue, namely S.

Fringe: { S }

Then, we add all of S's successors onto the queue:

Fringe: { SA SB SC }

Per the hint, we added, SA, SB, SC, in that order. The oldest item is therefore SA, so SA is the next item popped off the queue.

Fringe: { SA SB SC }

Then, we add all of SA's successors onto the queue:

Fringe: { SB SC SAC SAE }

The oldest item still on the queue is SB, so we'll pop off SB next.

Fringe: { SB SC SAC SAE }

Then, we'll add all of SB's successors onto the fringe:

Fringe: { SC SAC SAE SBA SBC }

The oldest item still on the queue is SC, so we'll pop off SC next.

Fringe: { SC SAC SAE SBA SBC }

Then, we'll add all of SC's successors onto the fringe. This is the first time we call the successor function on state C.

Fringe: { SAC SAE SBA SBC SCD }

The oldest item still on the queue is SAC, so we'll pop off SAC next.

Fringe: { SAC SAE SBA SBC SCD }

Then, we'll add all of SAC's successors onto the fringe. This is the second time we call the successor function on state C.

Fringe: { SAE SBA SBC SCD SACD }

The oldest item still on the queue is SAE, so we'll pop off SAE next.

Fringe: { SAE SBA SBC SCD SACD }

Then, we'll add all of SAE's successors onto the fringe.

Fringe: { SBA SBC SCD SACD SAEG }

The oldest item still on the queue is SBA, so we'll pop off SBA next.

Fringe: { SBA SBC SCD SACD SAEG }

Then, we'll add all of SBA's successors onto the fringe.

Fringe: { SBC SCD SACD SAEG SBAC SBAE }

The oldest item still on the queue is SBC, so we'll pop off SBC next.

Fringe: { SBC SCD SACD SAEG SBAC SBAE }

Then, we'll add all of SBC's successors onto the fringe. This is the third time we call the successor function on state C.

Fringe: { SCD SACD SAEG SBAC SBAE SBCD }

The oldest item still on the queue is SCD, so we'll pop off SCD next.

Fringe: { SCD SACD SAEG SBAC SBAE SBCD }

Then, we'll add all of SCD's successors onto the fringe.

Fringe: { SACD SAEG SBAC SBAE SBCD SCDE SCDG }

The oldest item still on the queue is SACD, so we'll pop off SACD next.

Fringe: { SACD SAEG SBAC SBAE SBCD SCDE SCDG }

Then, we'll add all of SACD's successors onto the fringe.

Fringe: { SAEG SBAC SBAE SBCD SCDE SCDG SACDE SACDG }

The oldest item still on the queue is SAEG, so we'll pop off SAEG next.

Fringe: { SAEG SBAC SBAE SBCD SCDE SCDG SACDE SACDG }

SAEG is a valid path to the goal, so we are done.

In total, we called the successor function on state C three times.

In **BFS Graph Search**, how many times does the successor function get called on state C?

1

Q4)

TRUE / FALSE

1. UCS graph search is guaranteed to find the optimal solution.

True, Uniform-cost graph search will consider all the paths with cost 0, then all the paths with cost 1, then all the costs with path 2, etc. until finding the optimal (least-cost) solution. (Note the assumption that all costs are positive, so we don't have to worry about negative-cost solutions.)

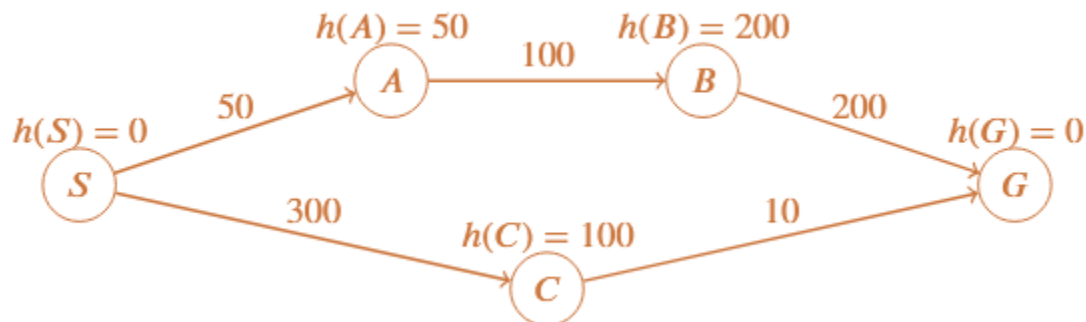
2. You design a heuristic function h such that for all states s , $h(s) > 0$. If you run greedy graph search on the problem using the heuristic, it is guaranteed to get the optimal solution.

False. Greedy search with an arbitrary heuristic is not guaranteed to get the optimal solution.

3. Suppose both UCS graph search and greedy graph search return the same optimal path for a problem. If you use the heuristic in 2, it is guaranteed that A* graph search will also return the optimal path.

False. Because if the heuristic is not consistent, then A* graph search is not guaranteed to be optimal. UCS and greedy search could return an optimal solution with an inadmissible or inconsistent heuristic (e.g. if there's only one solution). However, A* graph search is not guaranteed to return the optimal solution if the heuristic is not consistent.

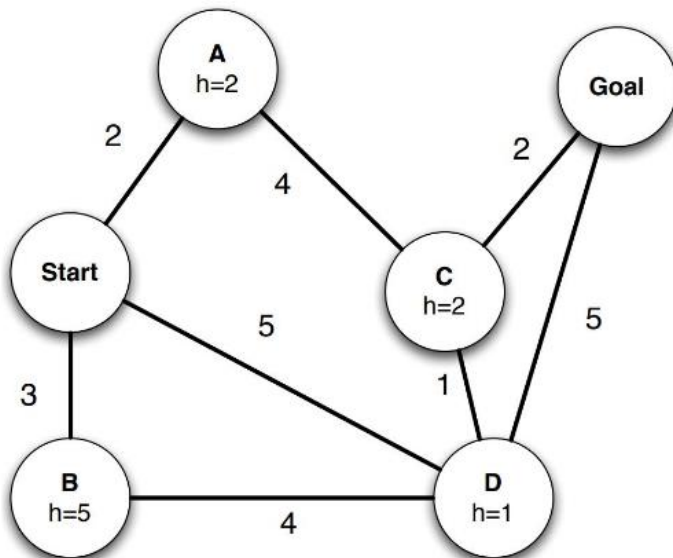
A counterexample here. The optimal path is S-C-G but A* will return S-A-B-G.



4. If a heuristic is consistent, it is also admissible.

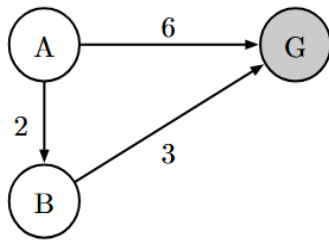
True

For each of the following graph search strategies, work out the order in which states are expanded, as well as the path returned by graph search. In all cases, assume ties resolve in such a way that states with earlier alphabetical order are expanded first. The start and goal state are S and G, respectively. Remember that in graph search, a state is expanded only once.



- a) Depth-first search.
 States Expanded: Start, A, C, D, B, Goal
 Path Returned: Start-A-C-D-Goal
- b) Breadth-first search.
 States Expanded: Start, A, B, D, C, Goal
 Path Returned: Start-D-Goal
- c) Uniform cost search.
 States Expanded: Start, A, B, D, C, Goal
 Path Returned: Start-A-C-Goal
- d) Greedy search with the heuristic h shown on the graph.
 States Expanded: Start, D, Goal
 Path Returned: Start-D-Goal
- e) A* search with the same heuristic.
 States Expanded: Start, A, D, C, Goal
 Path Returned: Start-A-C-Goal

For the following questions, consider the search problem shown on the left. It has only three states, and three directed edges. A is the start node and G is the goal node. To the right, four different heuristic functions are defined, numbered I through IV.



	$h(A)$	$h(B)$	$h(G)$
I	4	1	0
II	5	4	0
III	4	3	0
IV	5	2	0

For each heuristic function, circle whether it is admissible and whether it is consistent with respect to the search problem given above.

	Admissible?		Consistent?	
I	☒ Yes	☐ No	☒ Yes	☐ No
II	☐ Yes	☒ No	☒ Yes	☐ No
III	☒ Yes	☐ No	☒ Yes	☐ No
IV	☒ Yes	☐ No	☒ Yes	☐ No

II is the only inadmissible heuristic, as it overestimates the cost from B : $h(B) = 4$, when the actual cost to G is 3.

To check whether a heuristic is consistent, ensure that for all paths, $h(N) - h(L) \leq \text{path}(N \rightarrow L)$, where N and L stand in for the actual nodes. In this problem, $h(G)$ is always 0, so making sure that the direct paths to the goal ($A \rightarrow G$ and $B \rightarrow G$) are consistent is the same as making sure that the heuristic is admissible. The path from A to B is a different story.

Heuristic I is not consistent: $h(A) - h(B) = 4 - 1 = 3 \not\leq \text{path}(A \rightarrow B) = 2$.

Heuristic III is consistent: $h(A) - h(B) = 4 - 3 = 1 \leq 2$

Heuristic IV is not consistent: $h(A) - h(B) = 5 - 2 = 3 \not\leq 2$

Circle all true statements among the following.

- Heuristic function III dominates IV.
- Heuristic function IV dominates III.
- ☒ Heuristic functions III and IV have no dominance relationship.
- Heuristic function I dominates IV.
- ☒ Heuristic function IV dominates I.
- Heuristic functions I and IV have no dominance relationship.

For one heuristic to dominate another, *all* of its values must be greater than or equal to the corresponding values of the other heuristic. Simply make sure that this is the case. If it is not, the two heuristics have no dominance relationship.

For the following question parts, all edges in the graphs discussed have cost 1.

Suppose that you are designing a heuristic h for the graph on the right. You are told that $h(F) = 0.5$, but given no other information. What ranges of values are possible for $h(D)$ if the following conditions must hold? Your answer should be a range, e.g. $2 \leq h(D) < 10$. You may assume that h is nonnegative.

i. h must be admissible

$$0 \leq h(D) \leq 3$$

The path to goal from D is 3

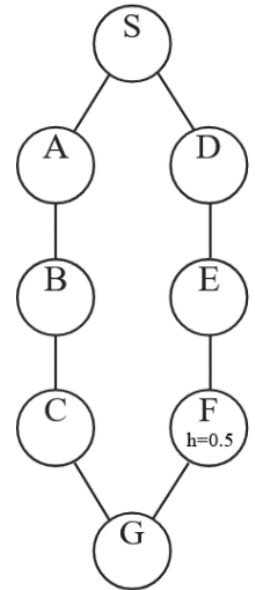
ii. h must be admissible and consistent

$$0 \leq h(D) \leq 2.5$$

In order for $h(E)$ to be consistent, it must hold that $h(E) - h(F) \leq 1$, since the path from E to F is of cost 1. Similarly, it must hold that:

$$h(D) - h(F) = h(D) - 0.5 \leq 2$$

$$\text{or } h(D) \leq 2.5.$$



For each of the following situations, pick the search that is most appropriate (be specific about visited and expanded list). Give a one sentence reason why you picked it. If you write a paragraph, we will not read it.

1. We have a very large search space with a large branching factor and with possibly infinite paths. We have no heuristic. We want to find paths to the goal with minimum numbers of state.

Iterative deepening is the best choice, it uses little memory (like DFS) but guarantees finding the path with minimum number of states (like BFS).

2. We have a space with a manageable number of states but lots of cycles in the state graph. We have links of varying costs but no heuristic and we want to find shortest paths.

Uniform Cost Graph Search is the best choice, it guarantees finding shortest paths and the expanded list limits the cost to a function of the number of states, which is reasonable in this case. Recall that a visited list will interfere with the correct operation of UCS.

3. Our search space is a tree of fixed depth and all the goals are the leaves of the tree. We have a heuristic and we want to find any goal as quickly as possible.

This has a typo which makes it ambiguous. If you read it as "all the leaves are goals", then depth-first search is the best choice (gets to the leaves fastest). If you read it as "all the goals are at the leaves", then the best choice is a greedy search (best first), which uses the heuristic to guide you to the part of the tree with the goals. In neither case is a visited or expanded list advisable since we are searching a tree (no loops).

4. We have a space with a manageable number of states but lots of cycles in the state graph. We have links of varying costs and an admissible heuristic and we want to find shortest paths.

This calls for A* with graph search. This allows us to use all the information we have and to avoid the extra cost due to cycles.